

Multiple Comparison - Problem Scenario

Problem Statement: Running more tests simultaneously increases chances of finding something significant by accident.

With $\alpha = 0.05$, false positive risk grows with each additional test.

- with 1 test, false positive risk is 5%
vs.
- with 5 tests, false positive risk is 23%

** false positive: incorrectly declaring a result significant when nothing real changed*

Tests	False Positive Risk
1	5%
2	10%
3	14%
4	19%
5	23%

Example: all primary and secondary metrics appear significant and the guardrail is safe — looks like a strong win, but our false positive risk is **23%**

Type	Metric	p-value	Significant? (no correction)	False Positive Risk
Primary	Revenue	0.012	✓ Yes	23%
Secondary	CTR	0.009	✓ Yes	
	Time Spent	0.028	✓ Yes	
	Page Views	0.045	✓ Yes	
Guardrail	Opt-out Rate	0.080	✗ No (Safe)	

Multiple Comparison - Correction Solutions

Solutions: To address multiple comparison, we adjust the significance threshold (α) for each test. There are 3 common approaches, choosing based on your business objective.

Method	Logic	Adjusted α (5 tests)	Best For
Bonferroni	Divide α equally	0.010 each	High stakes – multi-million dollar revenue calls, healthcare decisions, fraud detection
Holm-Bonferroni (HB)	Sequential adjustment	0.010, 0.013, 0.017, 0.025, 0.050	Most business experiments – product launches, campaign evaluation
Benjamini-Hochberg (BH)	Controls false discovery rate	0.010, 0.020, 0.030, 0.040, 0.050	Exploratory analysis – screening many features, early-stage hypothesis generation

Bonferroni Ex: High stakes decision → Primary not significant. Do not proceed.

HB Ex: Social supporting campaign → Primary significant, guardrail safe. Safe to continue.

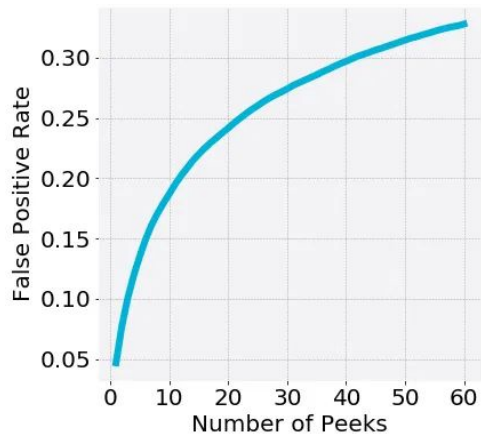
Type	Metric	p-value	No Correction	Bonferroni	HB	BH
Primary	Revenue	0.012	✓	✗	✓	✓
Secondary	CTR	0.009	✓	✓	✓	✓
	Time Spent	0.028	✓	✗	✗	✓
	Page Views	0.045	✓	✗	✗	✗
Guardrail	Opt-out Rate	0.080	✗ (Safe)	✗ (Safe)	✗ (Safe)	✗ (Safe)

* Correction methods applied in p-value rank order (smallest to largest)

The Peeking Problem

What is peeking? Checking experiment results before the planned end date and acting on what you see — declaring a winner and stopping early.

Why is it a problem? This **inflates your false positive rate** — e.g., peek 5 times and your false positive risk is close to 10%, not 5%. The more you peek, the more likely you are to see a 'significant' result by chance.



Source: Skyscanner Engineering

Solutions

1 Don't do it: Stick to the plan.

Let the experiment run to its pre-specified end date.

2 Peek safely, if you must: Purpose-built methods allow interim monitoring while keeping false positive rates in check.

- **Alpha Spending:** Pre-commit to a fixed look schedule.
- **Sequential Testing** (mSPRT): Peek any time.

Caveat: Safe peeking methods exist, but they set a much higher bar for early decisions.

The Peeking Problem - Alpha Spending Solution

When to Use: You know in advance when you'll check results.

How to Set it Up

1

Decide your look schedule
Choose # peek times and at what sample fractions, e.g. peek once at 50% of planned sample, once at 100%

2

Apply O'Brien-Fleming
Spends little alpha early and saves most of the budget for the final look. Overall error rate stays at 5%.

3

Compare at each look
Cross the threshold → stop early and declare a winner.
Don't cross it → continue to the next look

Learnings

- Early looks require a much stricter threshold. It takes an exceptionally strong signal to stop early.
- More looks make the final threshold harder to cross. **Two looks** is the industry sweet spot.

Ex: Look schedule	Look 1	Look 2	Look 3	Final look	Power cost
2 looks (50%, 100%)	$p < 0.005$	—	—	$p < 0.042$	Minimal ✓
3 looks (33%, 67%, 100%)	$p < 0.0006$	$p < 0.015$	—	$p < 0.034$	Moderate
4 looks (25%, 50%, 75%, 100%)	$p < 0.0001$	$p < 0.005$	$p < 0.017$	$p < 0.027$	Higher

Example: baseline 10%, MDE 2pp, $\alpha = 0.05$, power = 80%, total sample 7,600

The Peeking Problem - mSPRT Solution

When to Use: A more flexible approach to peek any time, or MDE is uncertain at design time.

How it Works

- At every peek, compute a mixture likelihood ratio (mLR). The higher it is, the more the data points to a real effect.
- Stop as soon as $mLR > 1/\alpha$ (e.g., 20 for $\alpha = 0.05$)
- The false positive guarantee holds at every peek.

Example: By peek 4, at just 80% of the planned sample, the mLR crosses 20, giving a statistically valid early stop.

Peek	Sample (n)	Control %	Test %	Z-stat	mLR	Decision
1	200 (20%)	20.1%	21.9%	0.69	2.21	Continue
2	400 (40%)	19.8%	22.2%	1.01	3.89	Continue
3	600 (60%)	20.2%	22.8%	1.38	7.14	Continue
4	800 (80%)	20.0%	23.5%	1.98	22.67 ✓	STOP – winner

Example: baseline 20%, $\alpha = 0.05$, power = 80%, $\sigma^2 = 0.01$, total sample 1,000

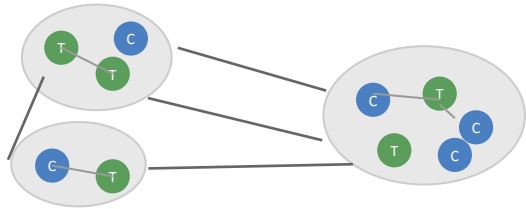
vs. SPRT: If your MDE is known upfront, SPRT stops faster when the true effect matches exactly. In practice, mSPRT is the better default — most business questions are "is there any effect?" rather than "did we hit a specific lift?"

Network Effects - Problem

Problem Statement: Standard A/B testing assumes users act independently (SUTVA). In connected networks, this breaks down: treated users influence control users through shared connections, causing treatment effects to leak across group boundaries and **understating the true lift**.

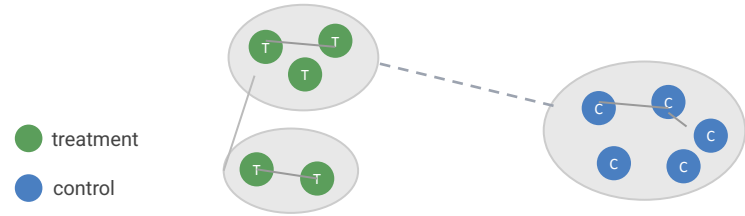
Unit Randomization

Mixed assignment — interference crosses group boundaries



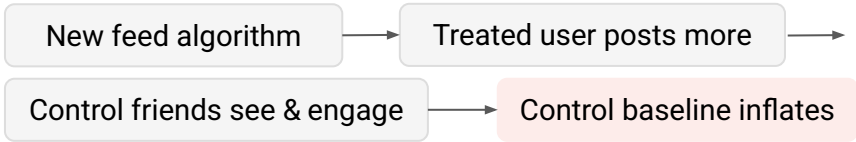
SOLUTION: Cluster Randomization

Whole clusters assigned to one arm — cross-cluster edges minimized



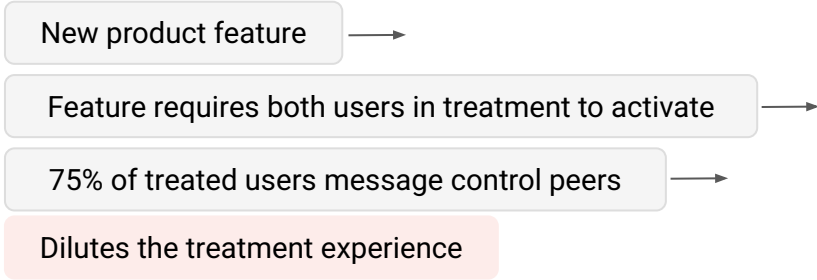
Problem: Spillover

Control baseline inflates, measured lift understates true effect
Ex: Facebook / Instagram / TikTok / LinkedIn



Problem: Under-Treatment

Feature rarely fires, effect diluted at the source
Ex: WhatsApp



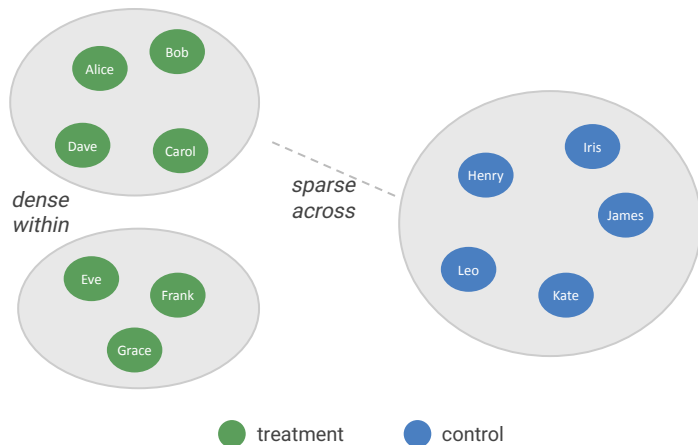
Network Effects - Solution

Cluster Randomization

- Use graph community detection (**Louvain**) to partition users into dense clusters with minimal cross-cluster edges.
- Randomize at the cluster level – spillover stays within arms, and treated users interact with other treated users

Method: Modularity Optimization (Louvain)

Algorithm: graph community detection



For each possible grouping, Louvain scores:

$$Q = \frac{(\text{actual edges within group})}{(\text{edges expected by chance})}$$

Higher Q = denser within-group connections. Louvain greedily merges nodes to maximize Q.

THE DESIGN TRADEOFF: POWER VS. PURITY

Goal: find the right cluster size for your product interaction density – enough purity to contain spillover, enough clusters to detect an effect.

More clusters, smaller size

- + More statistical power
- More cross-cluster edges, more spillover risk

Fewer clusters, larger size

- + Cleaner containment, less spillover
- Fewer independent units, wider confidence intervals